Aula 14 — Funções e modularização

Dividindo problemas em partes menores, reutilizando código e trabalhando com parâmetros e retorno

Dieimes Nunes

Retomando a Aula 13

Na aula anterior, começamos a representar registros usando dicionários.

Também criamos estruturas como:

```
produtos = [
    {"codigo": 1, "nome": "Mouse", "preco": 80.0},
    {"codigo": 2, "nome": "Teclado", "preco": 120.0}
]
```

Com isso, conseguimos criar cadastros mais próximos de sistemas reais.

Mas surgiu um novo problema:

> Nossos programas estão ficando grandes e repetitivos.

O problema da aula

Imagine que um programa precise calcular média várias vezes.

Podemos escrever:

```
media1 = (nota1 + nota2) / 2
```

Depois em outro trecho:

```
media2 = (nota3 + nota4) / 2
```

E novamente:

```
media3 = (nota5 + nota6) / 2
```

A lógica é a mesma.

Estamos repetindo código.

O que é modularização?

Modularizar significa dividir um problema maior em partes menores.

Em vez de pensar:

```
"Como vou construir o sistema inteiro?"
```

podemos pensar:

1. Como cadastrar?
2. Como calcular?
3. Como buscar?
4. Como listar?
5. Como validar?

Cada parte pode ser criada e testada separadamente.

Atividade 1 — Dividindo um problema

No caderno, pense em um sistema de biblioteca com:

```
Cadastrar livro
Listar livros
Buscar livro
Emprestar livro
Devolver livro
Sair
```

Divida o sistema em pequenas tarefas.

Para cada tarefa, responda:

1. Que dados ela recebe?
2. O que ela faz?
3. Ela precisa devolver algum resultado?
4. Poderia ser uma função?

O que é uma função?

Uma função é um bloco de código criado para realizar uma tarefa específica.

Exemplo:

```
def mostrar_mensagem():
    print("Bem-vindo ao sistema")
```

Criar a função não executa o código imediatamente.

Para executar:

```
mostrar_mensagem()
```

Isso é chamado de **chamada da função**.

Definição e chamada

Definição:

```
def saudacao():
    print("Olá, turma!")
```

Chamada:

```
saudacao()
```

Podemos chamar várias vezes:

```
saudacao()
saudacao()
saudacao()
```

O código da função é reutilizado.

Atividade 2 — Primeira função

Crie:

`primeiras_funcoes.py`

Implemente três funções:

```
mostrar_titulo()
mostrar_menu()
mostrar_despedida()
```

Cada função deve mostrar mensagens diferentes.

Depois faça chamadas na ordem correta para formar um pequeno programa.

Funções sem parâmetros

Exemplo:

```
def mostrar_menu():
    print("1 - Cadastrar")
    print("2 - Listar")
    print("0 - Sair")
```

A função não precisa receber informações para executar.

Ela sempre mostra o mesmo menu.

Esse tipo de função é útil para tarefas fixas.

Parâmetros

Às vezes a função precisa receber dados.

```
def saudacao(nome):
    print(f"Olá, {nome}!")
```

Chamada:

```
saudacao("Ana")
saudacao("Carlos")
```

O parâmetro nome recebe valores diferentes em cada chamada.

Parâmetro e argumento

Na definição:

```
def saudacao(nome):
```

`nome` é um **parâmetro**.

Na chamada:

```
saudacao("Ana")
```

`"Ana"` é o **argumento** enviado para a função.

É comum usar os termos juntos no início do aprendizado.

Atividade 3 — Função com parâmetro

Crie uma função:

```
def mostrar_aluno(nome, media):
```

Ela deve exibir:

```
Aluno: Ana
Média: 8.50
```

Faça pelo menos três chamadas com alunos diferentes.

Depois acrescente o parâmetro `frequencia`.

Mais de um parâmetro

Uma função pode receber vários dados.

```
def mostrar_produto(nome, preco, quantidade):
    print(f"Produto: {nome}")
    print(f"Preço: R$ {preco:.2f}")
    print(f"Quantidade: {quantidade}")
```

Chamada:

```
mostrar_produto("Teclado", 120.0, 5)
```

A ordem dos argumentos precisa corresponder aos parâmetros.

Funções que calculam

Podemos colocar uma expressão dentro da função:

```
def calcular_media(nota1, nota2):
    media = (nota1 + nota2) / 2
    print(media)
```

Isso funciona.

Mas existe uma diferença importante entre **mostrar** e **devolver** um resultado.

print() não é return

Compare:

```
def calcular_media(nota1, nota2):
    media = (nota1 + nota2) / 2
    print(media)
```

Essa função mostra o resultado.

Agora:

```
def calcular_media(nota1, nota2):
    media = (nota1 + nota2) / 2
    return media
```

Essa função **devolve** o resultado para quem fez a chamada.

Usando o valor retornado

```
def calcular_media(nota1, nota2):
    return (nota1 + nota2) / 2

resultado = calcular_media(8, 6)

print(resultado)
```

Podemos usar o retorno em outra lógica:

```
if resultado >= 7:
    print("Aprovado")
else:
    print("Reprovado")
```

Essa separação aumenta a reutilização.

Atividade 4 — Função com retorno

Crie:

`funcoes_calculo.py`

Implemente:

```
somar(a, b)
subtrair(a, b)
multiplicar(a, b)
dividir(a, b)
```

Todas devem retornar o resultado.

Na divisão, trate o caso de divisão por zero.

Depois crie um pequeno menu que use essas funções.

Procedimento

Em algumas ementas e linguagens, usamos a palavra **procedimento** para representar uma rotina que executa uma ação e não devolve um valor de cálculo.

Em Python, usamos funções para os dois casos.

Exemplo semelhante a um procedimento:

```
def mostrar_cabecalho():
    print("====================")
    print(" SISTEMA DE ESTOQUE ")
    print("====================")
```

Ela realiza uma ação, mas não precisamos de um `return` com resultado útil.

Atividade 5 — Retorno ou apenas ação?

Classifique cada tarefa.

A

Mostrar o menu principal.

B

Calcular a média de três notas.

C

Verificar se um número é par.

D

Mostrar uma linha de separação.

E

Calcular o total de uma compra.

Para cada uma, diga:

> Ela precisa retornar um valor? Por quê?

Função booleana

Uma função pode retornar `True` ou `False`.

```
def eh_par(numero):
    return numero % 2 == 0
```

Uso:

```
numero = int(input("Número: "))

if eh_par(numero):
    print("Par")
else:
    print("Ímpar")
```

A função responde uma pergunta lógica.

Função para validar

Podemos encapsular uma validação:

```
def nota_valida(nota):
    return 0 <= nota <= 10
```

Uso:

```
nota = float(input("Nota: "))

if nota_valida(nota):
    print("Nota válida")
else:
    print("Nota inválida")
```

A função deixa o código principal mais fácil de ler.

Atividade 6 — Funções booleanas

Crie funções:

```
eh_positivo(numero)
eh_maior_de_idade(idade)
nota_valida(nota)
senha_valida(senha)
```

Defina uma regra para `senha_valida`.

Exemplo:

```
ter pelo menos 6 caracteres
```

Teste cada função com diferentes valores.

Variáveis locais

Observe:

```
def calcular_total(preco, quantidade):
    total = preco * quantidade
    return total
```

A variável:

`total`

foi criada dentro da função.

Ela é uma **variável local**.

Seu uso está relacionado à execução daquela função.

Escopo de variável

Exemplo:

```
def teste():
    mensagem = "Olá"
    print(mensagem)

teste()
```

Dentro da função funciona.

Mas usar fora:

```
print(mensagem)
```

não funciona, pois `mensagem` foi criada localmente.

Esse conceito é chamado de **escopo**.

Atividade 7 — Prevendo o erro

Analise sem executar:

```
def calcular():
    resultado = 10 + 5
    print(resultado)

calcular()
print(resultado)
```

Responda:

1. A função executa?
2. O primeiro `print()` funciona?
3. O segundo `print()` funciona?
4. Por quê?
5. Como podemos devolver o resultado corretamente?

Depois corrija usando `return`.

Evitando variáveis globais desnecessárias

Programas iniciantes às vezes usam muitas variáveis fora das funções.

Exemplo:

```
total = 0
```

E várias funções alteram diretamente esse valor.

Isso pode dificultar entender quem modificou o dado.

Sempre que possível, prefira:

- receber dados por parâmetros;
- calcular dentro da função;
- devolver o resultado com `return`.

Uma função pode chamar outra função

Exemplo:

```
def calcular_media(n1, n2):
    return (n1 + n2) / 2


def situacao(media):
    if media >= 7:
        return "Aprovado"
    return "Reprovado"


media = calcular_media(8, 6)
print(situacao(media))
```

Dividimos responsabilidades.

Atividade 8 — Separando responsabilidades

Crie funções:

```
calcular_media(n1, n2, n3)
definir_situacao(media)
mostrar_resultado(nome, media, situacao)
```

O programa principal deve:

1. receber os dados;
2. chamar `calcular_media`;
3. usar o retorno em `definir_situacao`;
4. chamar `mostrar_resultado`.

Observe como cada função tem uma responsabilidade clara.

Funções trabalhando com listas

Podemos receber uma lista como parâmetro.

```
def calcular_media_lista(notas):
    soma = 0

    for nota in notas:
        soma += nota

    return soma / len(notas)
```

Uso:

```
notas = [8, 7, 9]
media = calcular_media_lista(notas)
print(media)
```

Cuidado com lista vazia

A função anterior teria problema se:

```
notas = []
```

Porque não podemos dividir por zero.

Podemos tratar:

```
def calcular_media_lista(notas):
    if len(notas) == 0:
        return None

    soma = 0

    for nota in notas:
        soma += nota

    return soma / len(notas)
```

Atividade 9 — Funções para listas

Crie funções:

```
somar_lista(valores)
maior_valor(valores)
menor_valor(valores)
media_lista(valores)
contar_pares(valores)
```

Não use inicialmente:

```
sum()
max()
min()
```

Aplique a lógica que já aprendemos.

Teste com uma lista preenchida pelo usuário.

Funções trabalhando com registros

Podemos receber um dicionário.

```
def mostrar_produto(produto):
    print(f"Código: {produto['codigo']}")
    print(f"Nome: {produto['nome']}")
    print(f"Preço: R$ {produto['preco']:.2f}")
```

Uso:

```
produto = {
    "codigo": 1,
    "nome": "Mouse",
    "preco": 80.0
}

mostrar_produto(produto)
```

Função para buscar registro

```
def buscar_produto(produtos, codigo):
    for produto in produtos:
        if produto["codigo"] == codigo:
            return produto

    return None
```

Uso:

```
resultado = buscar_produto(produtos, 2)

if resultado is not None:
    mostrar_produto(resultado)
else:
    print("Produto não encontrado")
```

A função de busca não precisa mostrar mensagens.

Ela devolve o registro encontrado.

Atividade 10 — Refatorando o sistema de estoque

Pegue a lógica do sistema de estoque da Aula 13.

Crie funções:

```
mostrar_menu()
cadastrar_produto(produtos)
listar_produtos(produtos)
buscar_produto(produtos, codigo)
alterar_preco(produtos)
entrada_estoque(produtos)
saida_estoque(produtos)
```

O programa principal deve ficar menor e mais fácil de ler.

O que significa refatorar?

Refatorar é reorganizar e melhorar o código sem necessariamente mudar o que o programa faz para o usuário.

Exemplo:

Antes:

```
um programa enorme com todas as instruções misturadas
```

Depois:

```
funções menores com responsabilidades bem definidas
```

O comportamento pode continuar igual.

A organização melhora.

Função principal

Uma organização possível:

```
def main():
    print("Iniciando sistema...")
    mostrar_menu()


main()
```

Em programas maiores, uma função principal ajuda a mostrar onde o fluxo começa.

Não é obrigatório para todo exercício simples, mas é uma prática útil.

Exemplo de programa modularizado

```
def somar(a, b):
    return a + b


def subtrair(a, b):
    return a - b


def mostrar_menu():
    print("1 - Somar")
    print("2 - Subtrair")
    print("0 - Sair")


def main():
    while True:
        mostrar_menu()
        opcao = int(input("Opção: "))
```

Atividade 11 — Lendo o fluxo

Analise o programa anterior e responda:

1. Qual função é chamada primeiro no final do arquivo?
2. Qual função apenas mostra opções?
3. Quais funções retornam valores?
4. Onde ocorre a repetição do menu?
5. Onde acontece a tomada de decisão?
6. O que aconteceria se `mostrar_menu()` fosse removida?

Depois execute e teste.

Bons nomes de funções

Prefira nomes que indiquem ação:

```
calcular_media
buscar_produto
mostrar_menu
validar_nota
cadastrar_cliente
listar_alunos
```

Evite nomes vagos:

```
f1
coisa
funcao
teste2
faz_algo
```

A função deve explicar sua responsabilidade pelo nome.

Uma função deve fazer tudo?

Evite funções gigantes que:

- recebem dados;
- fazem vários cálculos;
- pesquisam;
- imprimem relatórios;
- alteram cadastros;
- controlam o menu inteiro.

Em geral, é melhor separar responsabilidades.

Uma pergunta útil:

> Consigo explicar em uma frase o que esta função faz?

Se a resposta exigir uma lista enorme, talvez ela esteja fazendo coisas demais.

Atividade 12 — Melhorando uma função grande

Considere a descrição:

```
função sistema_aluno():
- lê nome;
- lê três notas;
- valida notas;
- calcula média;
- define situação;
- imprime resultado;
- pergunta se deseja outro aluno.
```

No caderno, proponha uma divisão em funções menores.

Não existe apenas uma resposta correta.

Explique sua organização.

Erros comuns nesta aula

Esquecer os parênteses na chamada

```
mostrar_menu
```

não é a mesma coisa que:

```
mostrar_menu()
```

Esquecer o return

```
def somar(a, b):
    resultado = a + b
```

A variável externa não recebe automaticamente `resultado`.

Indentação incorreta

O corpo da função precisa estar indentado.

Chamar antes de definir em fluxo inadequado

A organização do arquivo deve permitir que a definição seja conhecida antes da chamada ocorrer.

Testando funções separadamente

Uma grande vantagem da modularização é poder testar partes pequenas.

Exemplo:

```
print(calcular_media(10, 8))
print(calcular_media(5, 5))
print(calcular_media(0, 10))
```

Antes de integrar ao programa completo, verificamos se a função se comporta corretamente.

Isso prepara nosso próximo conteúdo sobre testes e depuração.

Onde estamos no curso

Agora já conseguimos combinar:

```
condições
repetições
listas
matrizes
registros
funções
```

As funções representam um passo importante para a **programação estruturada e modularizada**.

Ainda falta estudar:

Desafio final da aula

Crie:

```
sistema_alunos_modular.py
```

Use uma lista de registros.

Menu:

```
1 - Cadastrar aluno
2 - Listar alunos
3 - Buscar aluno
4 - Mostrar média da turma
5 - Mostrar aprovados
6 - Mostrar aluno com maior média
0 - Sair
```

Crie pelo menos estas funções:

```
mostrar_menu()
cadastrar_aluno(alunos)
listar_alunos(alunos)
```

```
buscar_aluno(alunos, matricula)
calcular_media_turma(alunos)
mostrar_aprovados(alunos)
```

Regras:

- matrícula não pode se repetir;
- notas devem ficar entre 0 e 10;
- cada aluno deve ser um dicionário;
- funções de cálculo devem retornar valores quando fizer sentido;
- evite repetir código.

Fechamento

Nesta aula, trabalhamos com:

- modularização;
- definição e chamada de funções;
- parâmetros e argumentos;
- `return`;
- procedimentos como funções sem retorno útil;
- valores booleanos retornados;
- escopo e variáveis locais;
- funções com listas e registros;
- refatoração;
- função principal;
- organização e reutilização de código.